

Worksheet-4: OPP in C++ Part-2



Submitted by – James Limbu (23085133)

Submitted to - Aashish Acharya

Tasks

Task 1: Solve the following programming problems:

1. STL Container Practice: Write a program using STL containers that: (40 marks)

1. Uses vector<string> to store names (5 Marks)
2. Uses map<string, int> to store age against each name (5 Marks)
3. Implements functions to:
 1. Add new name-age pair (10 marks)
 2. Find all people above certain age (10 marks)
 3. Sort and display names alphabetically (10 marks)

Code:

```
#include <iostream>
#include <vector>
#include <map>
#include <algorithm>
using namespace std;
void addPerson(vector<string>& names, map<string, int>& ages, string name, int age) {
    names.push_back(name);
    ages[name] = age;
}
void findPeopleAboveAge(map<string, int>& ages, int m_age) {
    cout << "\n||||||| - PEOPLE ABOVE AGE " << m_age << " -|||||||" << endl;
    for (auto person : ages) {
        if (person.second > m_age) {
            cout << person.first << " (" << person.second << " years old)" <<
endl;
        }
    }
}
void displaySortedNames(vector<string> names) {
    sort(names.begin(), names.end());
    cout << "\n||||||| - SORTED NAMES -|||||||" << endl;
    for (string name : names) {
        cout << name << endl;
    }
}
int main() {
    vector<string> names;
```

```

map<string, int> ages;
char choice;
do {
    string name;
    int age;
    cout << "\nEnter name: ";
    cin >> ws;
    getline(cin, name);

    cout << "Enter age of " << name << ": ";
    cin >> age;

    addPerson(names, ages, name, age);

    cout << "Do you want to add another person? (y/n): ";
    cin >> choice;

} while (choice == 'y' || choice == 'Y');

displaySortedNames(names);
int ageLimit;
cout << "\nEnter age to filter people above it: ";
cin >> ageLimit;
findPeopleAboveAge(ages, ageLimit);
return 0;
}

```

OutPuts:

```

Enter name: james
Enter age of james: 32
Do you want to add another person? (y/n): y

```

1. Output when user runs the code

```

Enter name: james
Enter age of james: 32
Do you want to add another person? (y/n): y

```

```

Enter name: david
Enter age of david: 40
Do you want to add another person? (y/n): n

```

2. output when user wants to add more people and not want to add more people

```
||||||| - SORTED NAMES - |||||  
david  
james
```

Enter age to filter people above it: 25

3. Output when user come out of that input. It provides option to filter people age.

```
||||||| - PEOPLE ABOVE AGE 25 - |||||  
david (40 years old)  
james (32 years old)
```

4. People above 25 age according to userinput.

2. Stack Problem: Implement a stack using arrays (not STL) that: (20 marks)

1. Has basic push and pop operations
2. Has a function to find middle element
3. Has a function to reverse only bottom half of stack
4. Maintain stack size of 10

```
#include <iostream>  
using namespace std;  
class Stack {  
private:  
    int arr[10];  
    int top;  
public:  
    Stack() {  
        top = -1;  
    }  
    bool isFull() {  
        return top == 9;  
    }  
    bool isEmpty() {  
        return top == -1;  
    }  
    void push(int value) {  
        if (isFull()) {  
            cout << "Stack is full!\n";  
        } else {  
            top++;  
            arr[top] = value;  
        }  
    }  
};
```

```

        arr[top] = value;
    }
}
void pop() {
    if (isEmpty()) {
        cout << "Stack is empty!\n";
    } else {
        top--;
    }
}
void peek() {
    if (isEmpty()) {
        cout << "Stack is empty!\n";
    } else {
        cout << "Top element is: " << arr[top] << endl;
    }
}
void findMiddle() {
    if (isEmpty()) {
        cout << "Stack is empty!\n";
    } else {
        int mid = top / 2;
        cout << "Middle element is: " << arr[mid] << endl;
    }
}
void reverseBottomHalf() {
    int size = top + 1;
    int mid = size / 2;

    for (int i = 0; i < mid / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[mid - 1 - i];
        arr[mid - 1 - i] = temp;
    }

    cout << "Bottom half reversed!\n";
}
void display() {
    if (isEmpty()) {
        cout << "Stack is empty!\n";
    } else {
        cout << "Stack (top to bottom): ";
        for (int i = top; i >= 0; i--) {
            cout << arr[i] << " ";
        }
    }
}

```

```

        cout << endl;}}
    }
};
int main() {
    Stack s;
    s.push(1);
    s.push(2);
    s.push(3);
    s.push(4);
    s.push(5);
    s.push(6);
    s.display();
    s.findMiddle();
    s.reverseBottomHalf();
    s.display();
    return 0;
}

```

Output:

```

Stack (top to bottom): 6 5 4 3 2 1
Middle element is: 3
Bottom half reversed!
Stack (top to bottom): 6 5 4 1 2 3

```

1. Output from the given code.

3. **Queue Problem: Implement a queue using arrays (not STL) that: (20 marks)**

1. Has basic enqueue and dequeue operations
2. Has a function to reverse first K elements
3. Has a function to interleave first half with second half
4. Handle queue overflow/underflow

```

#include <iostream>
using namespace std;
class Queue {
private:
    int arr[10];
    int frnt;
    int back;
    int count;
public:
    Queue() {
        frnt = 0;
        back = -1;
        count = 0;
    }
};

```

```

}
bool isFull() {
    return count == 10;
}
bool isEmpty() {
    return count == 0;
}
void enqueue(int value) {
    if (isFull()) {
        cout << "Queue Overflow!\n";
    } else {
        back = (back + 1) % 10;
        arr[back] = value;
        count++;
    }
}
void dequeue() {
    if (isEmpty()) {
        cout << "Queue Underflow!\n";
    } else {
        frnt = (frnt + 1) % 10;
        count--;
    }
}
void display() {
    if (isEmpty()) {
        cout << "Queue is empty!\n";
        return;
    }

    cout << "Queue (front to rear): ";
    for (int i = 0; i < count; i++) {
        int index = (frnt + i) % 10;
        cout << arr[index] << " ";
    }
    cout << endl;
}
void reverseFirstK(int k) {
    if (k > count || k <= 0) {
        cout << "Invalid K!\n";
        return;
    }
    int tmp[10];
    for (int i = 0; i < k; i++) {
        tmp[i] = arr[(frnt + k - 1 - i) % 10];
    }
}

```

```

    }
    for (int i = k; i < count; i++) {
        tmp[i] = arr[(frnt + i) % 10];
    }
    for (int i = 0; i < count; i++) {
        arr[(frnt + i) % 10] = tmp[i];
    }

    cout << "First " << k << " elements reversed!\n";
}

void interleaveQueue() {
    if (count % 2 != 0) {
        cout << "Need even number of elements to interleave!\n";
        return;
    }
    int half = count / 2;
    int tmp[10];
    for (int i = 0; i < half; i++) {
        tmp[i * 2] = arr[(frnt + i) % 10];
    }

    for (int i = 0; i < half; i++) {
        tmp[i * 2 + 1] = arr[(frnt + half + i) % 10];
    }
    for (int i = 0; i < count; i++) {
        arr[(frnt + i) % 10] = tmp[i];
    }
    cout << "Queue interleaved!\n";
}

};

int main() {
    Queue q;
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);
    q.enqueue(60);
    q.display();
    q.reverseFirstK(3);
    q.display();
    q.interleaveQueue();
    q.display();
    return 0;
}

```


Output:

```
Queue (front to rear): 10 20 30 40 50 60
First 3 elements reversed!
Queue (front to rear): 30 20 10 40 50 60
Queue interleaved!
Queue (front to rear): 30 40 20 50 10 60
```

3. Output When user runs the code.

4. Linked List Problem: Create a singly linked list (not STL) that: (20 marks)

1. Has functions to insert at start/end/position
2. Has a function to detect and remove loops
3. Has a function to find nth node from end
4. Has a function to reverse list in groups of K nodes

```
#include <iostream>
using namespace std;
class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};
class LinkedList {
private:
    Node* head;
public:
    LinkedList() : head(nullptr) {}
    void insertAtStart(int val) {
        Node* newNode = new Node(val);
        newNode->next = head;
        head = newNode;
    }
    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
            return;
        }
        Node* current = head;
        while (current->next) {
            current = current->next;
        }
    }
};
```

```

    }
    current->next = newNode;
}

void insertAtPosition(int val, int pos) {
    if (pos == 1) {
        insertAtStart(val);
        return;
    }
    Node* newNode = new Node(val);
    Node* current = head;
    for (int i = 1; i < pos-1 && current; i++) {
        current = current->next;
    }
    if (!current) {
        cout << "Invalid position!\n";
        return;
    }
    newNode->next = current->next;
    current->next = newNode;
}

void findNthFromEnd(int n) {
    if (!head) {
        cout << "List is empty!\n";
        return;
    }
    Node* first = head;
    Node* second = head;
    for (int i = 0; i < n; i++) {
        if (!first) {
            cout << "List is too short!\n";
            return;
        }
        first = first->next;
    }
    while (first) {
        first = first->next;
        second = second->next;
    }

    cout << n << "th node from end: " << second->data << endl;
}

void reverseGroups(int k) {
    head = reverseGroupHelper(head, k);
}

Node* reverseGroupHelper(Node* node, int k) {

```

```

    Node* current = node;
    Node* prev = nullptr;
    Node* next = nullptr;
    int count = 0;
    while (current && count < k) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
        count++;
    }
    if (next) {
        node->next = reverseGroupHelper(next, k);
    }
    return prev;
}

void detectAndRemoveLoop() {
    if (!head || !head->next) {
        cout << "No loop found.\n";
        return;
    }
    Node* slow = head;
    Node* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            removeLoop(slow);
            cout << "Loop found and removed.\n";
            return;
        }
    }

    cout << "No loop found.\n";
}

void removeLoop(Node* loopNode) {
    Node* ptr1 = head;
    Node* ptr2 = loopNode;
    while (ptr1->next != ptr2->next) {
        ptr1 = ptr1->next;
        ptr2 = ptr2->next;
    }
    ptr2->next = nullptr;
}

void display() {

```

```

        Node* current = head;
        while (current) {
            cout << current->data << " -> ";
            current = current->next;
        }
        cout << "NULL\n";
    }
};

int main() {
    LinkedList list;
    list.insertAtEnd(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtStart(0);
    list.insertAtPosition(10, 3);
    cout << "Original list: ";
    list.display();
    cout << "2nd from end: ";
    list.findNthFromEnd(2);
    cout << "Reversing in groups of 2: ";
    list.reverseGroups(2);
    list.display();
    cout << "Checking for loops: ";
    list.detectAndRemoveLoop();
    return 0;
}

```

Output:

```

Original list: 0 -> 1 -> 10 -> 2 -> 3 -> NULL
2nd from end: 2th node from end: 2
Reversing in groups of 2: 1 -> 0 -> 2 -> 10 -> 3 -> NULL
Checking for loops: No loop found.

```